



Designing Artificial Intelligence Products

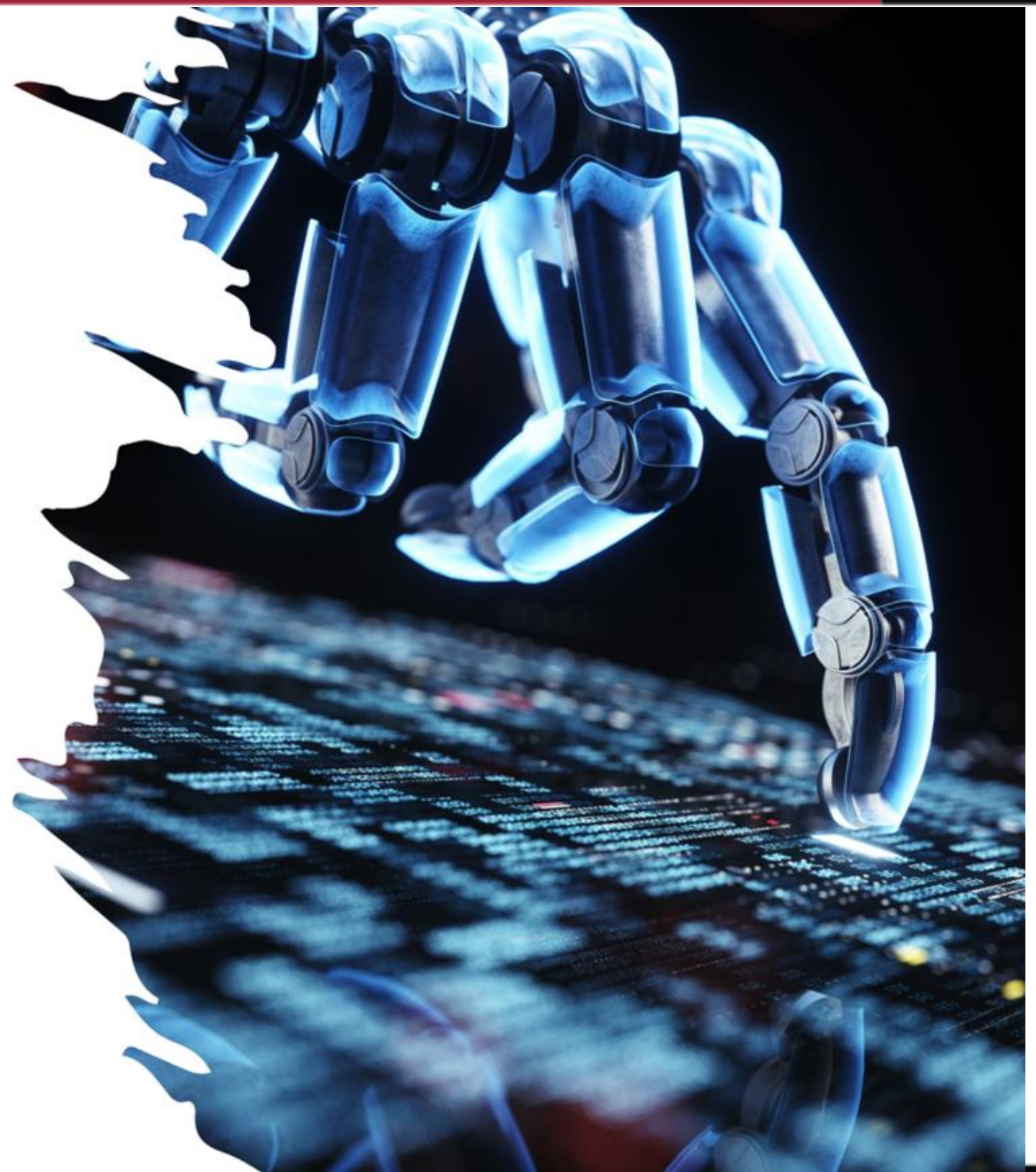
Module 2: Artificial Intelligence Technology
Fundamentals: Machine Learning

September 4, 2026

Dr. Amanda Fetch

Learning Outcomes

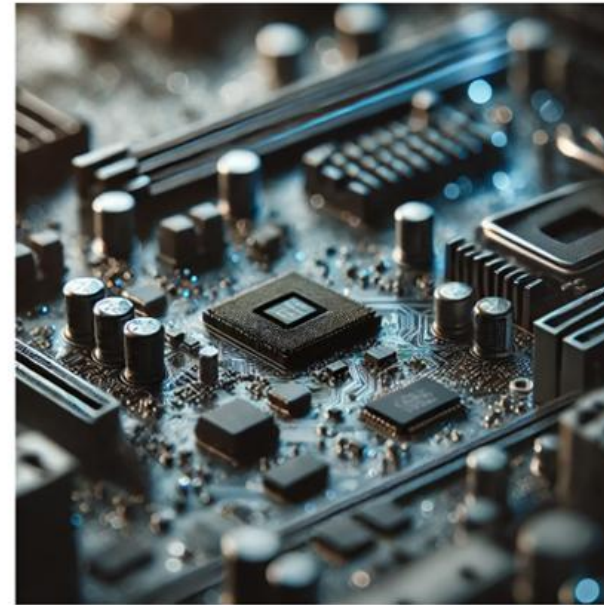
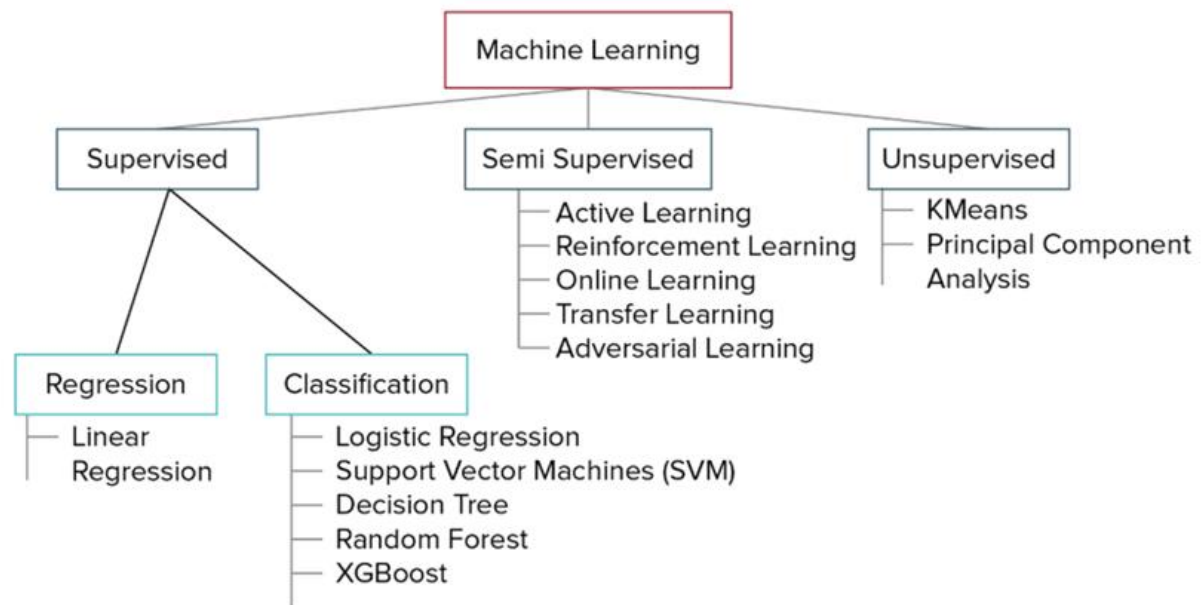
- Analyze the models, tools, and algorithms of machine learning.
- Classify points using a linear model.
- Identify the concepts of the train, validate, test methodology.
- Run machine learning algorithms in Python.





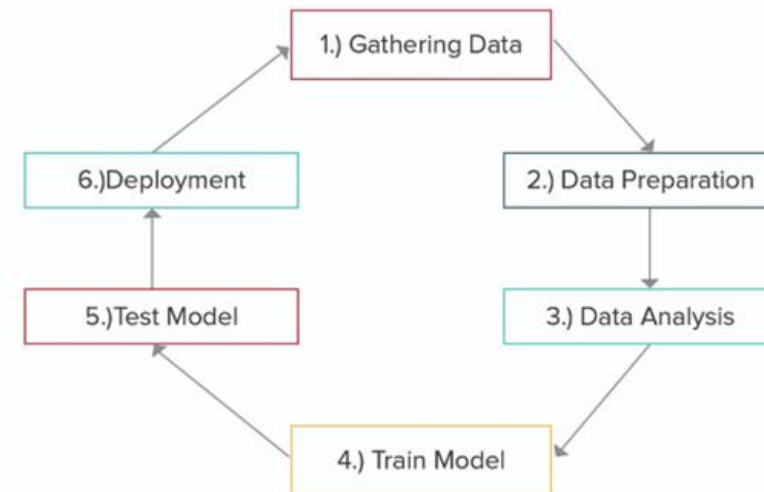
This week, we'll dive into the fascinating world of machine learning algorithms, exploring their core concepts and real-life applications. As we progress, you'll see how these algorithms power everyday technologies and solve complex problems. By the end of our discussion, you will have a comprehensive set of examples that provide a clear overview to reinforce your understanding.

Note: Some algorithms can be applied to the same use cases, as data science and machine learning involve a process of trial and error to determine which algorithm works best for a specific problem.





ML Lifecycle



Basic Concepts and Terminologies

Overfitting

Occurs when the model is too complex. Captures noise and irrelevant details from the training data. High accuracy on training data but poor performance on new, unseen data. Needs regularization or model simplification to mitigate.

Underfitting

Results from a model being too simplistic. Fails to capture the underlying patterns in the data. Low accuracy on both training and validation data. Requires a more complex model or feature engineering to improve.

Bias and Variance

Bias: Error from overly simplistic models; leads to underfitting.
Variance: Error from complex models that fit the training data too closely; leads to overfitting.
The goal is to balance bias and variance for optimal model performance.

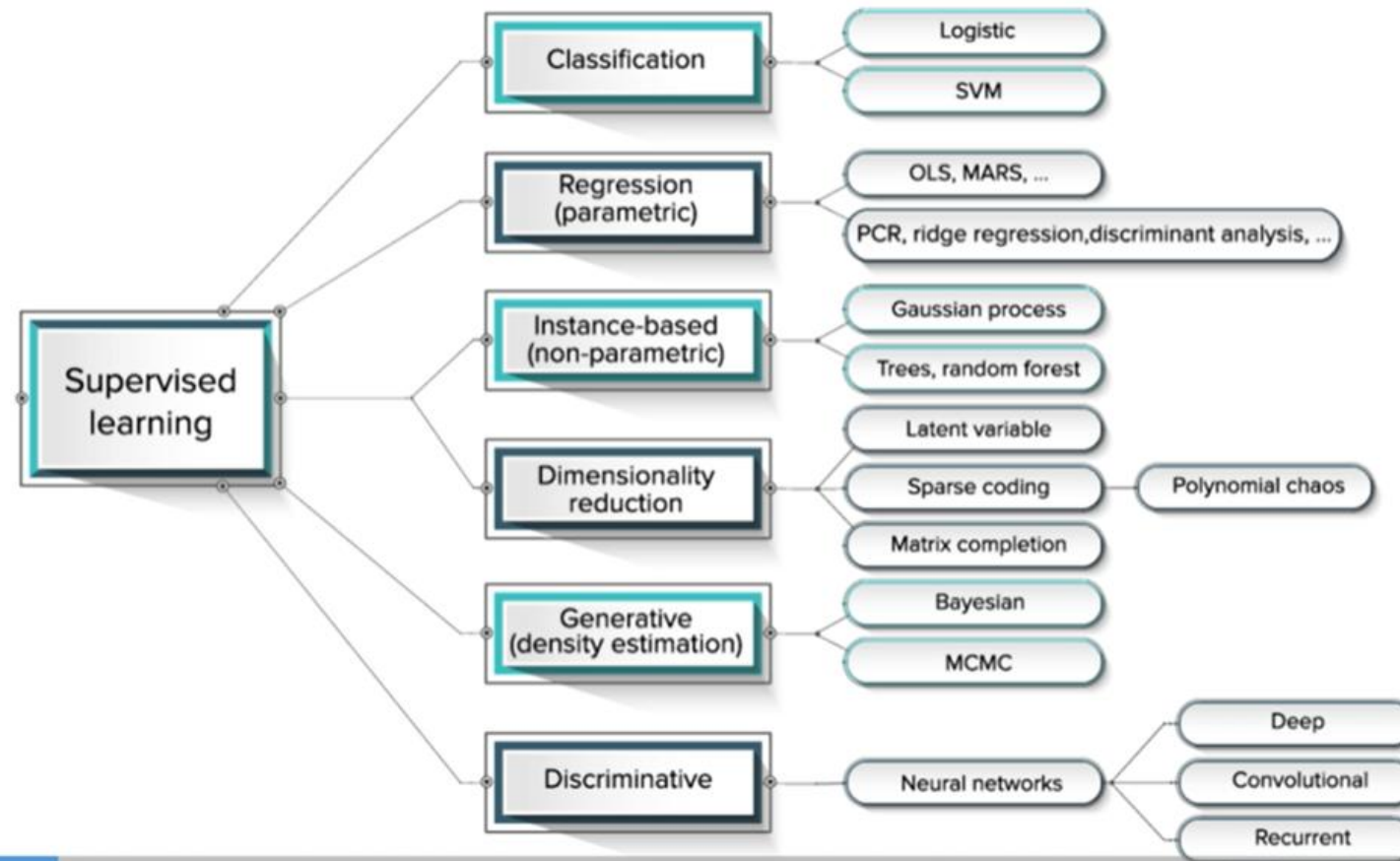
Bias-Variance Trade-Off

The challenge is finding a balance where both bias and variance are minimized. Reducing bias usually increases variance, and vice versa. Techniques like cross-validation help in managing this trade-off.

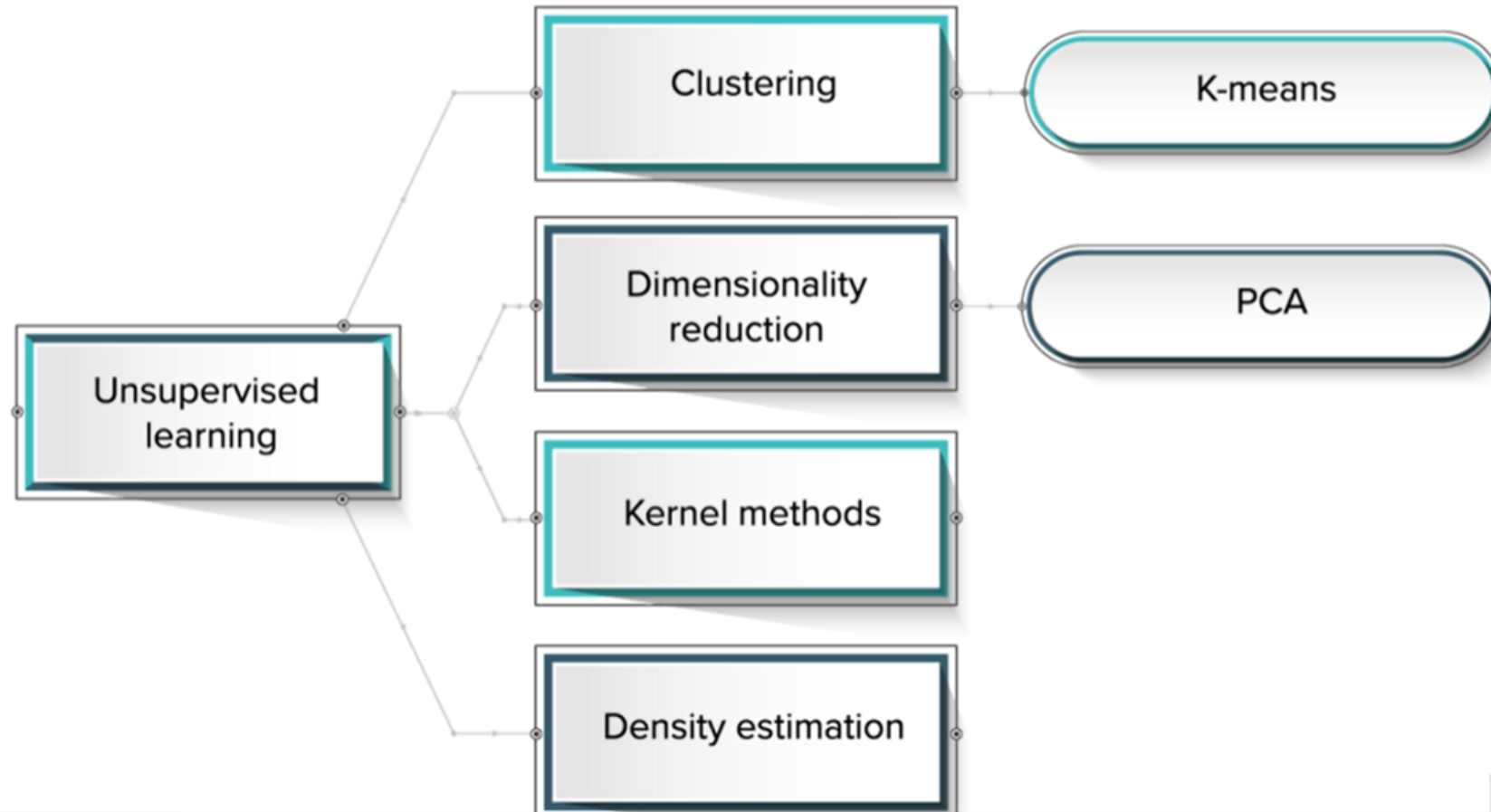
Hyperparameters and Tuning

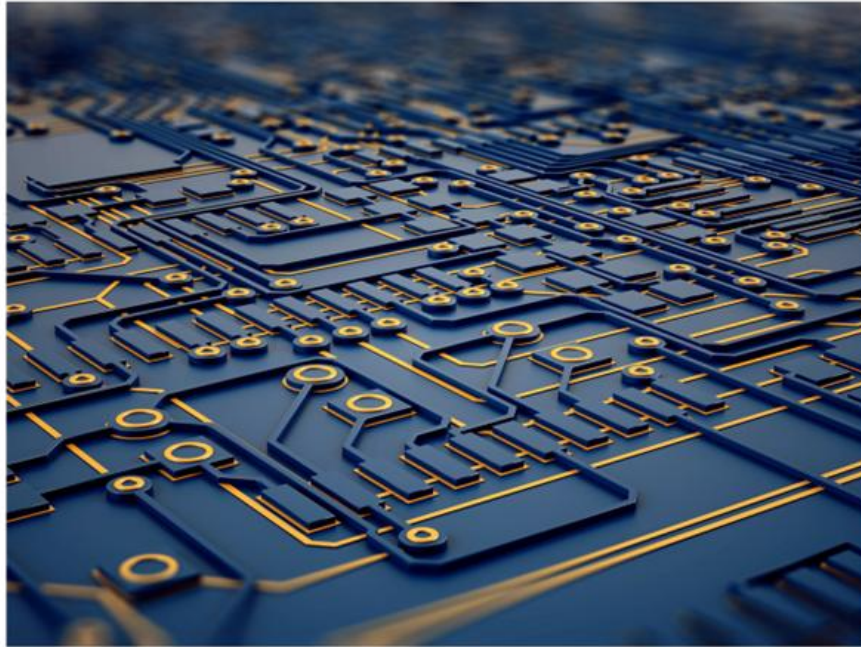
Hyperparameters control the learning process of the algorithm. Proper tuning is critical for model performance. Techniques like grid search, random search, and Bayesian optimization help in finding the best hyperparameter settings.

(Partial) Taxonomy of Machine Learning

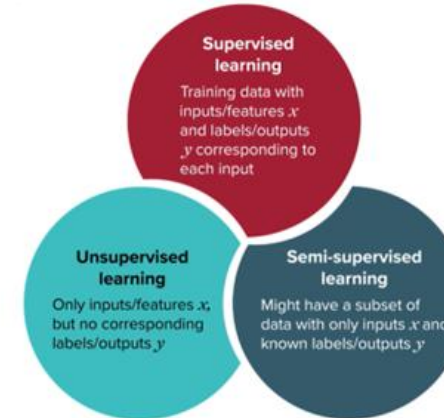


(Partial) Taxonomy of Machine Learning





(Partial) Taxonomy of Machine Learning



MIT xPRO

Types of Machine Learning Models

Parametric Models		Non - Parametric Models		Discriminative Models		Generative Models		Generative Adversarial Networks (GANs)	
Explanation	Use Cases	Explanation	Use Cases	Explanation	Use Cases	Explanation	Use Cases	Explanation	Use Cases
These models have a set number of parameters that don't change with the amount of data. They assume a specific form for the function mapping inputs to outputs. Example Models: Linear Regression, Logistic Regression, OLS	Predicting Housing Prices: Linear regression can predict house prices based on fixed factors like square footage and number of bedrooms. Email Spam Detection: Logistic regression helps classify emails as spam or not based on a fixed set of features.	Flexible Model Complexity: These models can adapt to the data more flexibly as they don't assume a fixed number of parameters. Example Models: Decision Trees, k-Nearest Neighbors (k-NN), Random Forests.	Customer Segmentation: k-NN can classify new customers into existing segments based on purchasing patterns. Medical Diagnosis: Random Forests can help predict diseases based on patient history and symptoms.	Focus on directly mapping inputs to outputs, predicting outcomes based on input features, and are often referred to as forward models. Ex. Neural networks, including deep, convolutional, and recurrent	Image Classification: Neural networks are used to classify images in apps like Google Photos. Speech Recognition: Deep Neural Networks are used in applications like virtual assistants.	Learn Data Distribution: These models focus on modeling class distributions and generating new data points. Example Models: Bayesian Models for probabilistic reasoning under uncertainty; Markov Chain Monte Carlo (MCMC) for approximating parameter distributions in complex models.	Medical Diagnosis: Bayesian models estimate disease risk based on patient symptoms and test results. Climate Modeling: MCMC methods simulate and predict future climate scenarios by estimating environmental variables.	Adversarial Learning: Combines a generative model with a discriminative model in a game-theoretic scenario to generate realistic data. Example Models: Standard GAN, StyleGAN, CycleGAN.	Synthetic Data Generation: GANs are used to generate synthetic data for training machine learning models, particularly when real data is scarce or sensitive, such as in healthcare or autonomous driving simulations. Deepfake Creation: GANs are behind many deepfake technologies, generating realistic faces or voices.

Self-Study Discussion 2.1: Using the Tools of Machine Learning [20 minutes]

Learning Outcome Addressed

1. Analyze the models, tools, and algorithms of machine learning.

This is a self-study discussion and does not count toward the completion of the program.

Instructions

In this module, Professor Subirana has discussed the applications of AI in healthcare. Three tools — regression analysis, tree-based models, and support vector machines — were used for the example. In this discussion, you will describe another example of how these tools have been used to create an AI application.

Task

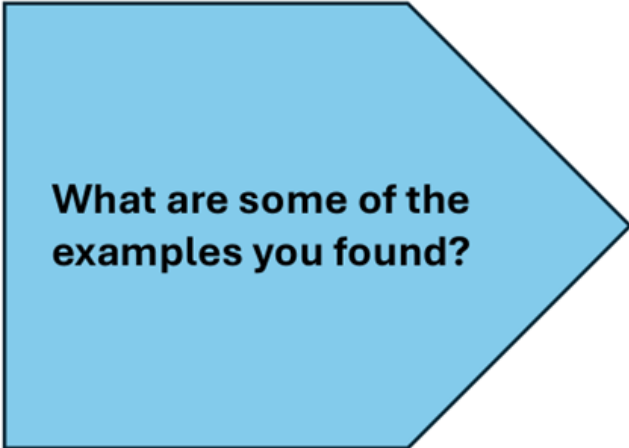
1. Locate an article where one or more tools have been used to create an AI application.
 - Regression analysis
 - Tree-based models
 - Support vector machines
2. Describe the application and the machine-learning tool used.
3. Describe how you might use this application for other industries or uses.

Prompts:

Read the statements posted by your peers. Feel free to respond to your peers with thoughtful comments and questions to deepen the discussion.

Submission Instructions

- The suggested word count for this discussion is 150-200 words.
- The average completion time for this activity is 20 minutes.



What are some of the examples you found?

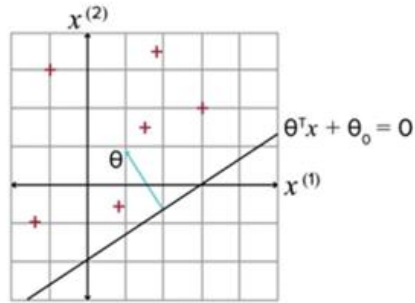
Linear Classification

- Consider an input (vector) feature space x
- Assume we have two classes:
 $y \in \text{"Positive" (+) or "Negative" (-)}$
- A linear classifier: A linear separator with model parameters

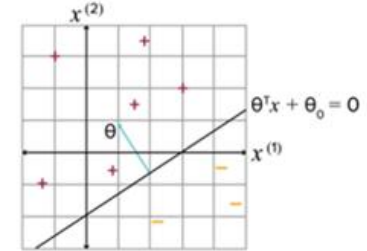
$$\theta^T x + \theta_0 = 0$$

$$\begin{bmatrix} -2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 6 = 0$$

- Inference: Decide "+" if $\theta^T x + \theta_0 > 0$ else "-"

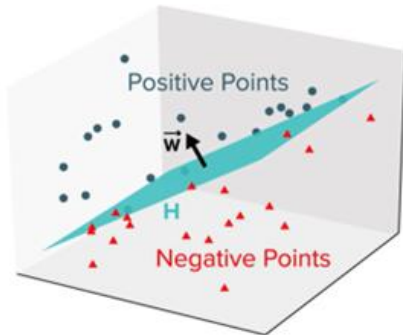


- Training occurs with some algorithm that uses some set of training data to find the coefficients θ^T and θ_0

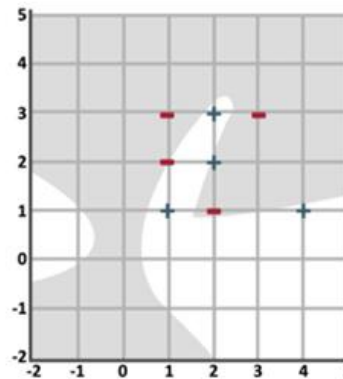


MIT

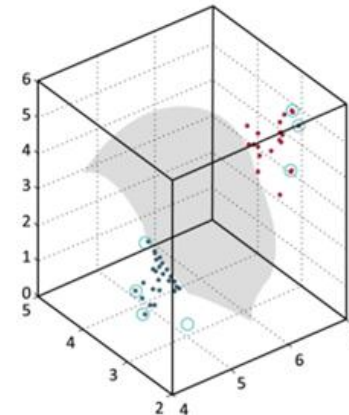
MIT



Transformation
 $f(x) = x_1, x_2$
Enables a simple linear classifier for the data



A higher order $f(x)$ enables a linear classifier for the data



A higher order $f(x)$ enables a complex linear classifier for the data

MIT

Linear Classification

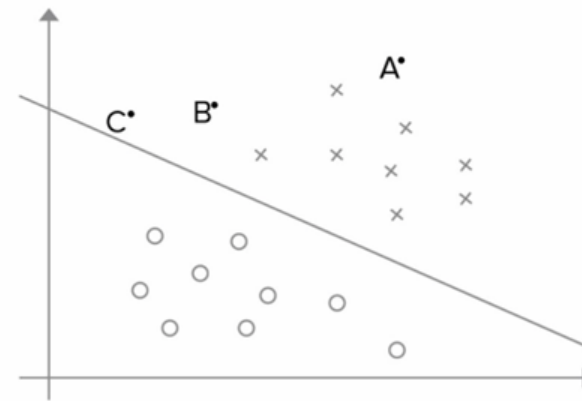
Consider the simplest linear classifier in two dimensions: a simple two-dimensional case with two features x_1 and x_2 .

The decision boundary is a line defined by:

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

Where w_0, w_1, w_2 are the y coefficient and coefficients of x_1 and x_2 , respectively. The equation of the straight line represents the linear decision boundary. Using the vector notation, the decision boundary line in this case is given by:

$$w_0 + w^T X = 0$$



College Admission

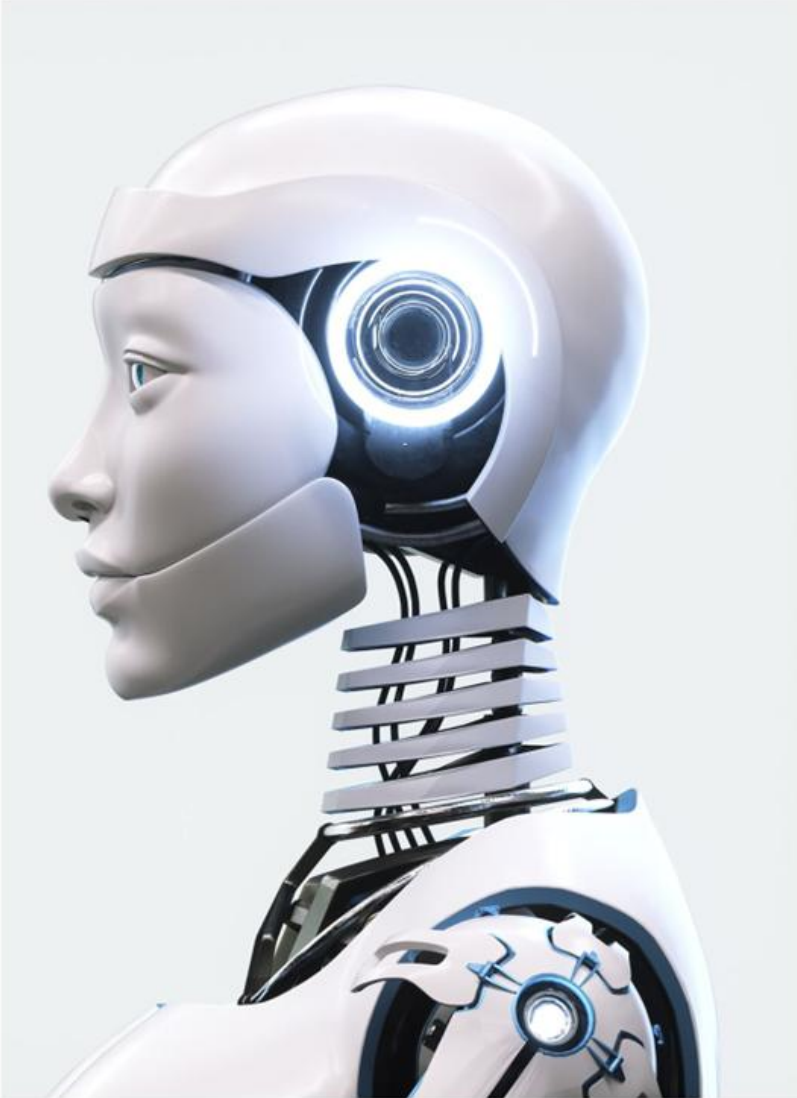


Credit Risk Assessment



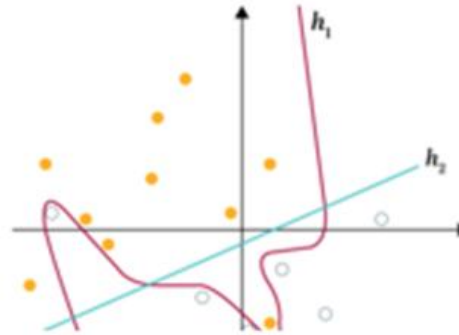
Customer Churn



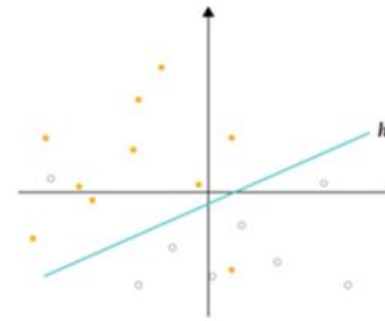


How Good is the Model?

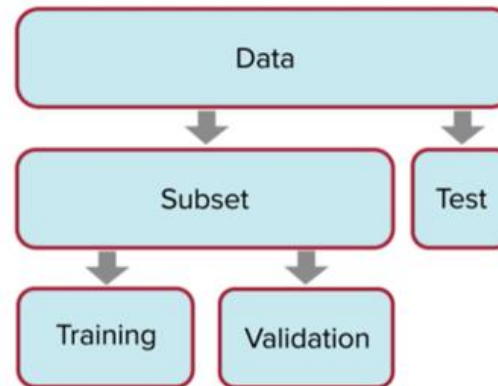
h_1 model



h_2 model

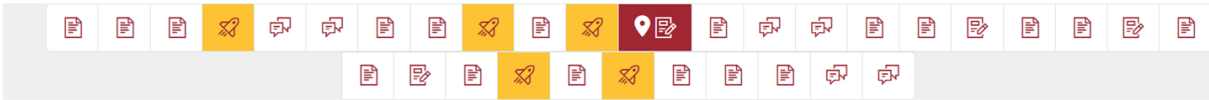


Training, Testing, and Validation Sets



- Do not use it during training
- Use it as a stand-in to evaluate error on unseen data

Assignment 2.1-Training and Testing



 Learning Outcome Addressed

4. Run machine learning algorithms in Python.

This is a self-study activity and does not count toward the completion of the program.

Task

You have learned how to separate training and testing datasets. In this exercise, you will practice this skill using Python. If this is your first experience with Python, this simple exercise will help you to become familiar with its structure and language.

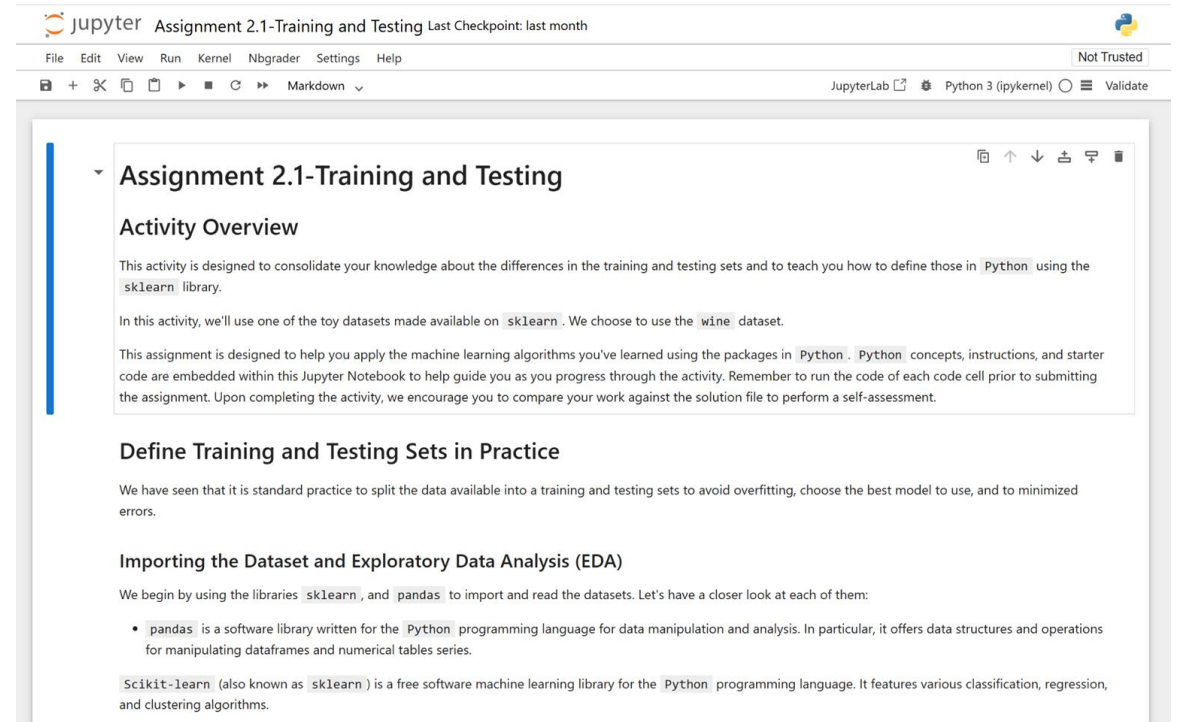
Submission Instructions

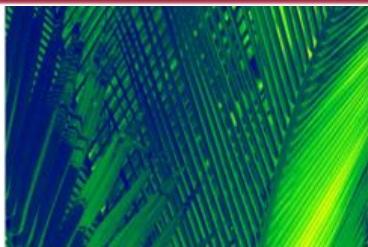
The average completion time for this activity is 30 minutes.

- After opening the activity in EMOdE, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in EMOdE or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#) . Find a [guide to viewing your solution in Google Colab](#)  here.
 - Download the solution here: [Assignment 2.1-Training and Testing](#)

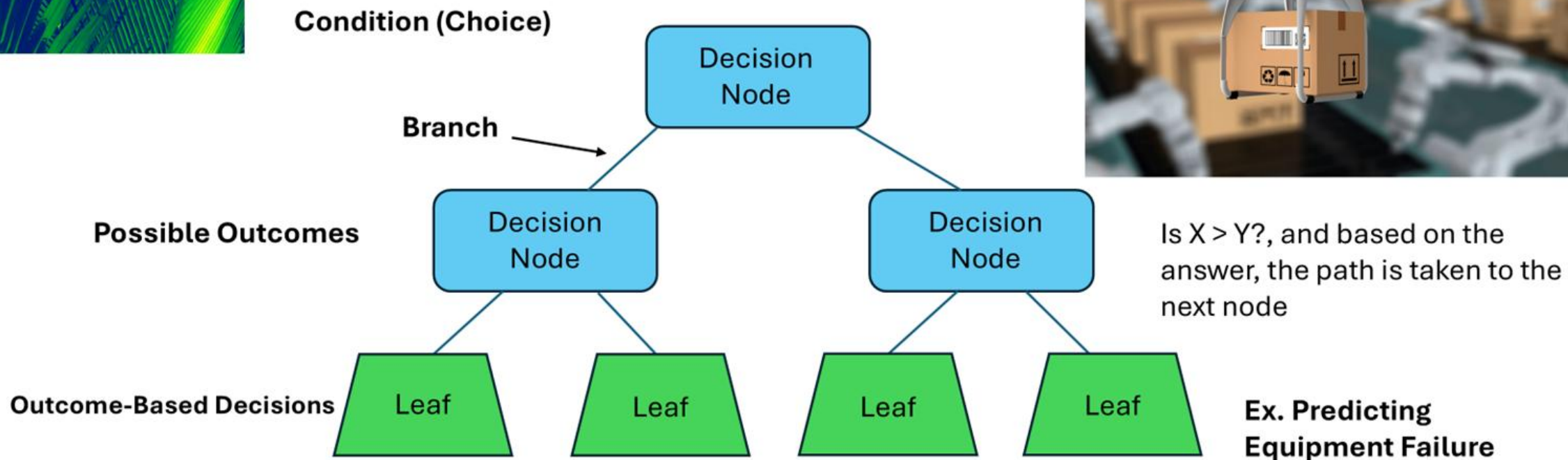
This tool needs to be loaded in a new browser window

Load Assignment 2.1-Training and Testing in a new window



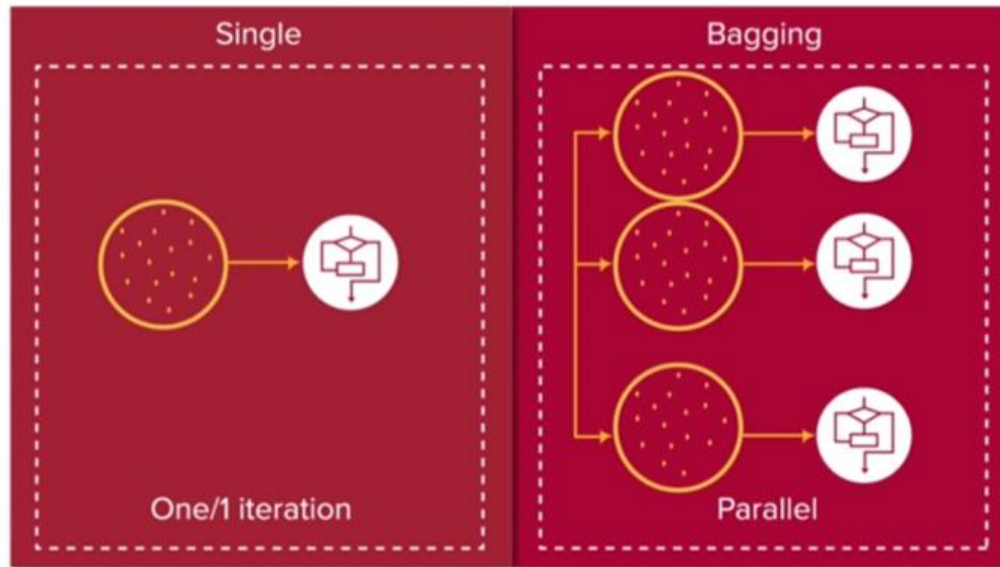


Decision Trees



Decision Node	Questions about the data
Branch	Possible outcomes of the condition at each decision node either leading to another decision node or leaf node
Leaf	Final decision or outcome after all the conditions have been evaluated

Bootstrap Aggregation or Bagging



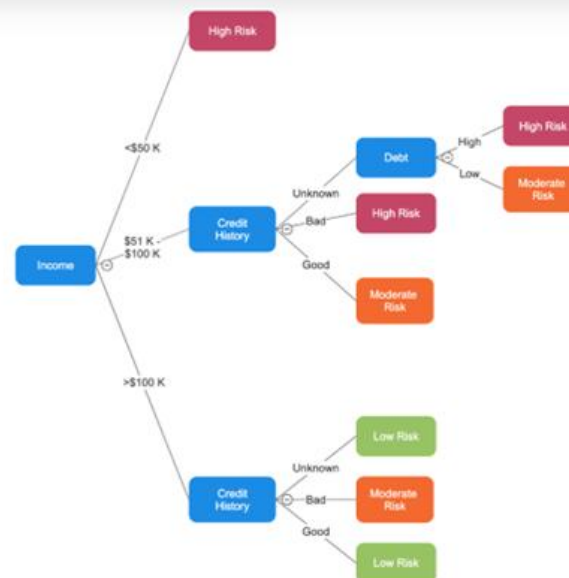
"What is the difference between Bagging and Boosting?" Quantdare. April 20, 2016.
[.com/what-is-the-difference-between-bagging-and-boosting/](https://quantdare.com/what-is-the-difference-between-bagging-and-boosting/)



Decision Tree

Generate decision trees quickly from data or a built-in template

Create Your Decision Tree >



Seven Tips for Creating a Decision Tree

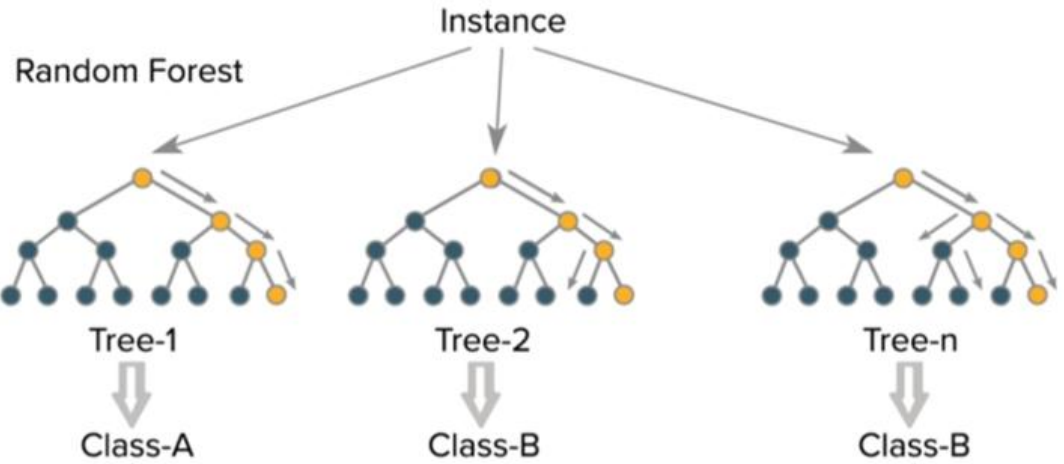
Here are some best practice tips for creating a decision tree diagram:

- **Start the tree.** Draw a rectangle near the left edge of the page to represent the first node. In this rectangle, write the first question, main idea, or criterion that will lead to a decision.
- **Add branches.** For every possible alternative draw a separate line that begins at the node and moves away toward the right of the page. Using a loan approval process as an example, the first node may have been "Income", and the associated branches might be "<\$50K", "\$51K - \$100K", ">\$101K".
- **Add leaves.** The bulk of the decision tree will be leaf nodes. At the end of each branch add a leaf node. Fill each of these leaf nodes with another question or criterion.
- **Add more branches.** Repeat the process of adding a branch for each possible alternative leading from a leaf. Label each branch just as before.
- **Complete the decision tree.** Continue adding leaves and branches until every question or criterion has been resolved and an outcome has been reached.
- **Terminate a branch.** Continue adding leaves and branches until every question or criterion has been resolved and an outcome has been reached.
- **Verify accuracy.** Consult with all stakeholders to verify accuracy.

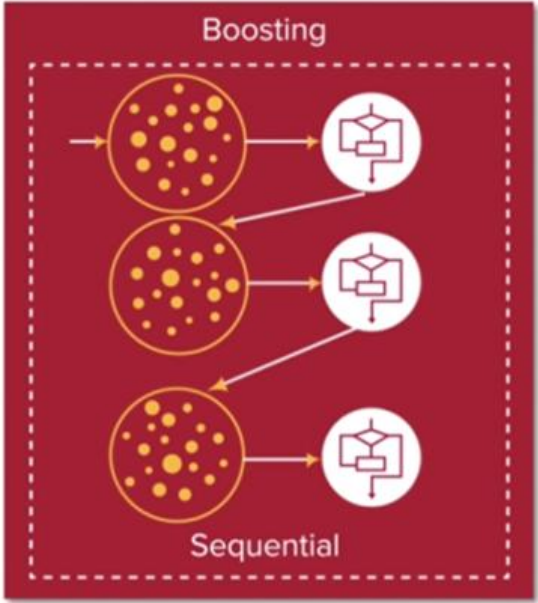
[↑ Back to top](#)

<https://www.smartdraw.com/decision-tree/>

Random Forest Machine Learning Model



Boosting



Bagging and Boosting? Quantdare. April 20, 2016.
[reen-bagging-and-boosting/](#)

Ex. Inventory
Optimization



Self-Study Try-It Activity 2.1: Construct a Tree-Like Diagram From a Dataset [10 minutes]

Learning Outcome Addressed

1. Analyze the models, tools, and algorithms of machine learning.

This is a self-study activity and does not count toward the completion of the program.

Context

Decision tree analysis is a predictive modeling tool with many applications. Decision trees are constructed using an algorithmic approach identifying how to split a dataset based on different conditions.

- A decision tree is a tree-like graph with node decisions.
- The edges represent the answers to the decision.
- Leaves represent the output or class label. They're used in nonlinear decision making with simple linear decision surfaces.

Assume you want to play golf on Saturday. How will you decide whether to play or not?

You can go outside on Saturday and observe the weather. You would observe the wind, the sky, the humidity, and the temperature.

However, you need to know before Saturday so you can make plans. You could calculate all of these factors for the last ten days and create a lookup table like the one below.

Table depicting daily weather conditions, including cloudiness, temperature, humidity, and wind strength

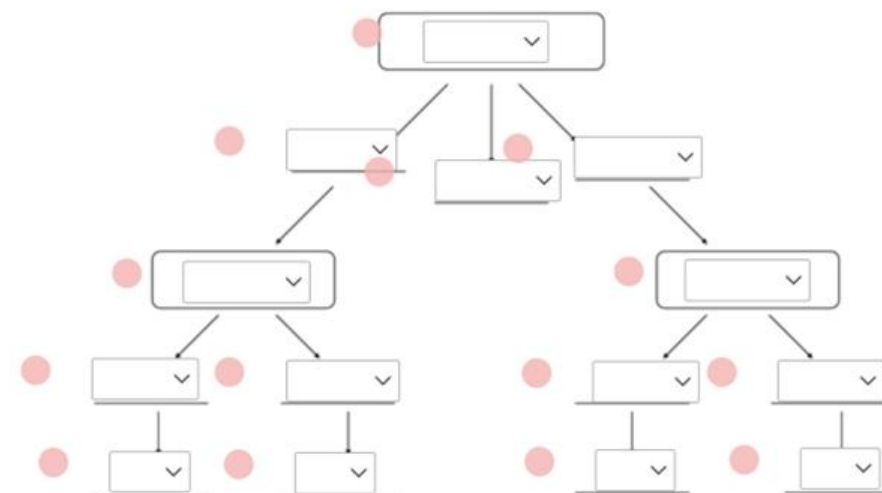
Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

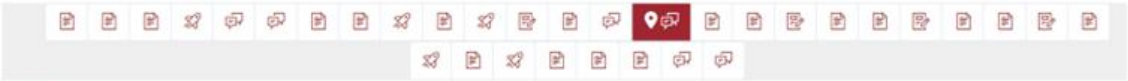
Task

Now you can use this table to decide whether to play. What if the weather pattern on Saturday doesn't match any of the table's rows?

A decision tree can represent this type of data because it considers all of the possible paths that can lead to the final decision by following a tree-like structure.

The average completion time for this activity is 10 minutes.





Learning Outcome Addressed

1. Analyze the models, tools, and algorithms of machine learning.

This is a self-study discussion and does not count toward the completion of the program.

Instructions

So far in this module, Professor Boning has discussed differentiating between linear classifiers and decision trees. In this activity, you will be given a simple dataset and asked to conclude the best tool for each dataset.

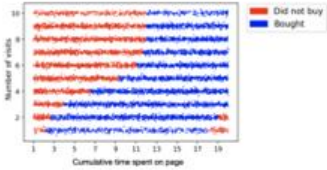
Suppose you've been keeping track of how many times individual customers visit this page, the total amount of time they've spent on the page across all their visits, and whether they bought the product. Using this based on the page visitation data, your goal is to make a prediction of how likely future visitors are to buy your AI product.

After collecting data, you create the following dataset:

Table depicting number of visits, total time on a web page, and whether the customer made a purchase.

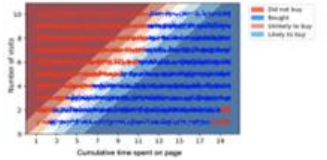
Number of Visits	Total Time (minutes)	Bought?
3	3	Yes
4	9	Yes
1	2	No
1	3	Yes
2	5	No
4	11	No
3	8	Yes
5	8	No
7	15	Yes

As an expert in data science and AI products, you've decided to run two different models on the data to help you make a better decision. You decide to run a linear classifier model and a decision tree model.



Linear Classifier:

At a high level, linear classifiers will take the feature space (i.e., the two-dimensional space where time is on the x-axis and the number of visits is on the y-axis) and seek to draw a straight line somewhere that creates an accurate division between the two classes of the response variable (i.e., "Bought" or "Did not buy").



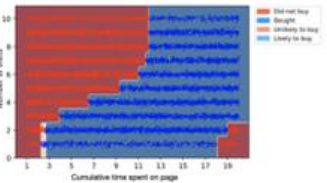
The regions are predicted by overlaying an area above and below the boundary line, with stronger likelihood at higher levels of visits and time spent. The conceptual straight-line decision boundary that divides the two regions mentioned above is where the probability of belonging to either class is very low. In other words, the model is "uncertain" about what prediction to make in this region.

Decision Tree Classifier:

How can you do better? Consider using a non-linear model.

Decision trees are a prime example of non-linear models. Decision trees work by dividing the data up into regions based on the "if-then" type of questions.

For example, if a user spends less than three minutes over two or fewer visits, how likely are they to buy? By asking many of these types of questions, a decision tree can divide up the feature space using little segments of vertical and horizontal lines. This approach can create a much more complex decision boundary.



It should be clear that decision trees can be used with more success to model this dataset. Given this information, you'd have a better model for the likelihood of customer conversion and could then proceed to design offers to increase conversion. What conclusion can you draw? Consider using a non-linear model.

Task

Your discussion must be based on the following pointers:

1. **Linear classifier:** Consider how likely this is to work. Where would you draw a straight line on the graph above so that the two regions on either side of the line would primarily contain responses of only one class?
2. **Decision tree classifier:** What conclusion can you draw?
3. What are the benefits and drawbacks of using the linear classifier versus the decision tree classifier?

Prompts:

Read the statements posted by your peers. Feel free to respond to your peers with thoughtful comments and questions to deepen the discussion.

Submission Instructions

- The suggested word count for this discussion is 100-150 words.
- The average completion time for this activity is 20 minutes.

Bayesian approach

- Enables use of prior experience and knowledge
- Inherently helps handle uncertainty
 - Dealing with noisy measurements in this approach
- Blends with adaptive or active learning methods
 - Make a choice about the next data point to acquire

Bayesian Methods

- For example, Gaussian (or Normal) distribution with mean μ and variance σ^2 : $\theta \sim N(\mu, \sigma^2)$
- Generate an updated *posterior belief* given a new observation y_i : $pdf(\theta | y_i)$
- Utilize a model of the *likelihood* of observing y_i given θ : $pdf(y_i | \theta)$
- As per Bayes' rule, the update can be achieved as:
$$pdf(\theta | y_i) \propto pdf(\theta) \cdot \prod_{i=1}^n pdf(y_i | \theta)$$

Bayesian Methods



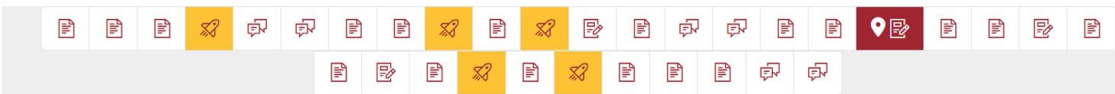
The probability of event A given that event B has occurred is equal to the probability of event B given that event A has occurred, multiplied by the probability of event A, and then divided by the probability of event B.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

- $P(A|B)$ is the probability of event A, given that event B has occurred. —→ Given that it's cloudy (event B), what's the chance it's going to rain (event A)?
- $P(B|A)$ is the probability of event B g, given that event A has occurred. —→ The chance of seeing clouds if we know it's going to rain
- $P(A)$ and $P(B)$ are the probabilities of events A and B respectively. —→ The probability of A happening before you even consider B. So, before you look at the clouds, how likely is rain based on your general knowledge (like weather patterns)? The probability of B happening on its own - how likely it is that you would see clouds, regardless of whether it's going to rain or not.

Assignment 2.2-Bayesian Inference



 Learning Outcome Addressed

4. Run machine learning algorithms in Python.

This is a self-study activity and does not count toward the completion of the program.

Task

You have learned about the purpose and mechanics of Bayesian machine learning algorithms. In this activity, you will practice running a Bayesian machine learning algorithm in Python and analyzing the results.

Submission Instructions

The average completion time for this activity is 30 minutes.

- After opening the activity in EMOdE, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in EMOdE or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#) . Find a [guide to viewing your solution in Google Colab](#)  here.
 - Download the solution here: [Assignment 2.2-Bayesian Inference](#)

This tool needs to be loaded in a new browser window

Load Assignment 2.2-Bayesian Inference in a new window

Jupyter Assignment 2.2-Bayesian Inference Last Checkpoint: last month

File Edit View Run Kernel Nbgrader Settings Help

Python 3 (ipykernel) Validate

Assignment 2.2-Bayesian Inference

Activity Overview

The activity is designed to consolidate the learner's knowledge about Bayesian methods and to teach them how to use these methods in Python with the `sklearn` library.

The session will begin with a simple, from scratch, implementation of the algorithm, and will conclude with a demonstration of how to execute the same model in `sklearn`.

The purpose of this activity is to assist the learner in applying the machine learning algorithms they have acquired using packages in Python. Python concepts, instructions, and starter code are embedded within the Jupyter Notebook to guide the learner as they progress through the activity. It is important for the learner to run the code in each code cell before submitting the assignment. After completing the activity, it is recommended that the learner compares their work against the solution file for self-assessment.

Conditional Probability Model and Classification

In machine learning, one is often interested in a predictive modeling problem where the goal is to predict a class label for a given observation. For instance, one might classify the species of plants based on the measurements of the flowers.

Such problems are known as classification predictive modeling problems. The observation or input to the model is referred to as X , and the class label or output of the model is referred to as y .

Together, X and y represent observations collected from the domain, that is, a table or matrix (columns and rows or features and samples) of training data used to fit a model. The model must learn how to map specific examples to class labels, or in other words, find a function $y = f(X)$, which minimizes the misclassification.

One approach to solving this problem is to develop a **probabilistic model**. From a probabilistic perspective, the conditional probability of the class label is estimated, given the observation.

Regression



Predicting Housing Prices

$$J(\theta) = \left[\frac{1}{n} \sum_{i=1}^n L(h(x^{(i)}; \theta), y^{(i)}) \right] + \lambda \cdot R(\theta)$$

where L is a loss function and R is a regularizer that penalizes model complexity

- Regularize by penalizing θ

Ridge Regression



Predicting Stock Prices

$$J(\theta, \theta_0) = \left[\frac{1}{n} \sum_{i=1}^n (\theta^T x^{(i)} + \theta_0 - y^{(i)})^2 \right] + \lambda \|\theta\|^2$$

Examples of how you have used these models?

Ordinary Least Squares (OLS)



Effect of study time on exam scores

- Minimizes the mean squared error over the n data points (without a regularizer)
- Direct (analytic) solution is possible

$$J_{OLS}(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^n [\theta^T x^{(i)} + \theta_0 - y^{(i)}]^2$$

Stochastic gradient descent (SGD)

Randomly pick individual training data points or batches of training points and iteratively update the model using those subsets.

Gradient Descent (GD)



Optimizing Delivery Routes

- An alternative perspective is to employ **gradient descent (GD)**

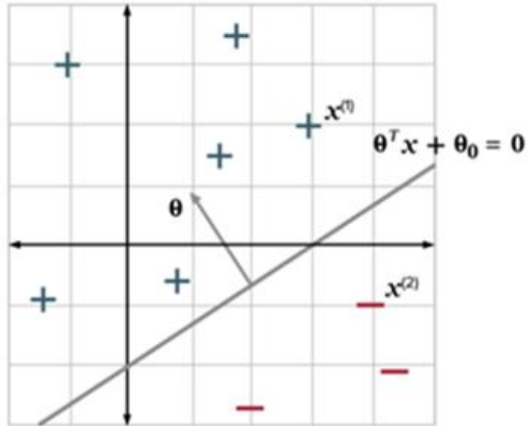
$$\nabla_{\theta} J = \frac{2}{n} \sum_{i=1}^n (\theta^T x^{(i)} + \theta_0 - y^{(i)}) x^{(i)} + 2\lambda \theta$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} J$$



Object Recognition in Images

Logistic Regression



- Consider an input (vector) feature space x
- Envision output y as the *probability* of membership in the class with label $y = 1$ vs. in the class with label $y = 0$
- Linear function (as in linear regression) for input x :

$$Z = \theta^T x + \theta_0$$

- Add a nonlinearity $\sigma(z)$ that “squashes” z to the range $[0, 1]$ indicating guess g as probability of class membership:

$$g = \sigma(z) = \frac{1}{1 + e^{-z}}$$

MIT

- Define the negative *log-likelihood* loss function between a guess $g^{(i)}$ for data point and the actual class $y^{(i)}$:
 $(g^{(i)}, y^{(i)}) = y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)})$
- Optimize to minimize this error (to penalize large θ):

$$J(\theta, \theta_0) = \left(\frac{1}{n} \sum_{i=1}^n L_{\text{hll}}(g^{(i)}, y^{(i)}) \right) + \lambda \|\theta\|$$

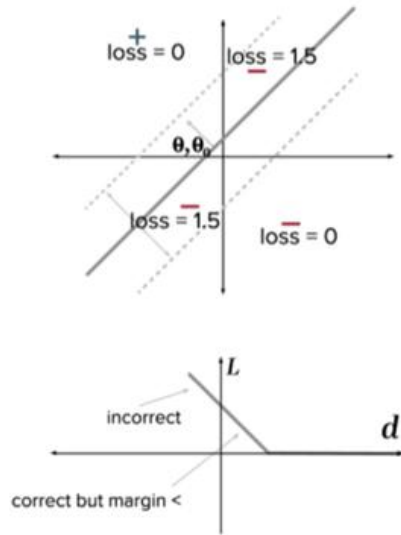
Inference: typically decide class = 1 if $g > 0.5$
 else class = 0

Real-World Example: Email Spam Detection

Imagine you want to build a model that predicts whether an incoming email is spam or not.

- **Features (x):** You could use features like the presence of certain words (like "win" or "free"), the sender's email address, or the length of the email.
- **Linear Function (Z):** You'd start by giving each of these features a weight based on how important they are in determining spam. For example, if the word "win" is very common in spam emails, it might get a high weight.
- **Sigmoid Function and Probability:** The result is passed through the sigmoid function to get a probability. For example, an email containing the word "win" and from an unknown sender might result in a probability of 0.85 (or 85%) of being spam.
- **Prediction:** If the probability is above 0.5, your model predicts that the email is spam; if it's below 0.5, it predicts it's not spam.

Support Vector Machines or SVMs



- As in logistic regression, we now optimize to minimize the loss (and to penalize large θ), but now with L_h rather than L_{nll} :

$$J(\theta, \theta_0) = \left(\frac{1}{n} \sum_{i=1}^n L_h(x^{(i)}, y^{(i)}) \right) + \lambda \|\theta\|$$

Inference: decide class = 1 if $d > 0$
else class = 0

Ex.
Handwriting
Recognition



Comparative Analysis of Classification and Regression Algorithms

Feature	Classification	Regression
Output Type	Produces discrete, categorical labels (e.g., yes/no decisions, class labels). Suitable for datasets where the result is limited to specific categories.	Produces continuous numerical outputs (e.g., price, temperature). Ideal for predictions where the result can vary widely and is not limited to specific categories.
Common Algorithms	Logistic regression, support vector machines (SVM), decision trees, random forests, and naive Bayes. These algorithms are designed to handle data that can be easily segmented into distinct classes.	Linear regression, polynomial regression, and ridge/lasso regression. These methods are capable of modeling complex relationships by fitting a continuous curve to the data.
Evaluation Metrics	Focuses on classification accuracy, precision (positive predictive value), recall (sensitivity), F1 score, and the use of a confusion matrix to visualize the performance of the classifier across different classes. These metrics help in understanding the effectiveness of classification in correctly labeling each class.	Utilizes mean squared error (MSE), mean absolute error (MAE), and root mean squared error (RMSE) to measure the average error magnitude without considering direction. These metrics are crucial for evaluating how close the predicted values are to the actual values.
Use Cases	Widely used in applications where binary or multiclass sorting is required, such as email spam detection, medical diagnosis, and image classification. Classification is crucial in scenarios where discrete decisions need to be made.	Common in economics, finance, and other fields that require precise numerical forecasts, such as predicting stock prices, real estate values, and weather conditions. Regression fits naturally into contexts where the output is a quantity rather than a category.
Data Sensitivity	Generally more robust against outliers because the primary goal is to categorize inputs correctly rather than predict exact values. Misclassified points can often be handled through robust training methods or by adjusting decision thresholds.	More sensitive to outliers because the accuracy of the prediction depends heavily on the range and distribution of the dataset. Outliers can significantly skew the regression model, leading to poor performance on typical data.
Complexity	Depending on the complexity of the data and the number of classes, classification models can be straightforward (e.g., binary classification) or complex (multiclass problems). The complexity often influences the choice of algorithm and the feature engineering required.	Can involve complex data transformations and tuning, especially when dealing with non-linear relationships. Polynomial features, normalization, and regularization are common techniques used to improve regression model performance and handle overfitting.

Conclusion

The choice between classification and regression is determined by the nature of the output variable (categorical vs. continuous) and the specific requirements of the application. Understanding these differences is essential for selecting the right algorithm and achieving high-quality predictive performance in various domains.

Assignment 2.3-Logistic Regression versus SVM



 Learning Outcome Addressed

4. Run machine learning algorithms in Python.

This is a self-study activity and does not count toward the completion of the program.

Task

You have learned about the purpose and mechanics of logistic regression and SVM models. In this activity, you will practice running these algorithms in Python and analyzing the results.

Submission Instructions

The average completion time for this activity is 30 minutes.

- After opening the activity in EMOde, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in EMOde or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#). Find a [guide to viewing your solution in Google Colab](#) ↓ here.
 - Download the solution here: [Assignment 2.3-Logistic Regression versus SVM](#)

This tool needs to be loaded in a new browser window

Load Assignment 2.3-Logistic Regression versus SVM in a new window

Jupyter Assignment 2.3-Logistic Regression versus SVM

File Edit View Run Kernel Nbgrader Settings Help

Not Trusted

JupyterLab No Kernel Validate

Assignment 2.3-Logistic Regression versus SVM

Activity Overview

This activity is designed to consolidate the knowledge about logistic regression and support vector machines (SVMs) from a practical point of view.

In particular, the user will try to diagnostically predict whether a patient has diabetes by using the Python libraries pandas and sklearn . They will begin by reading the dataset diabetes from Kaggle.

The activity aims to help the user apply machine learning algorithms using the packages in Python. Python concepts, instructions, and starter code are embedded within a Jupyter Notebook to guide them as they progress through the activity. It's essential to run the code of each cell before submitting the assignment. Upon completing the activity, the user is encouraged to compare their work against the solution file for self-assessment.

Logistic Regression and Support Vector Machines

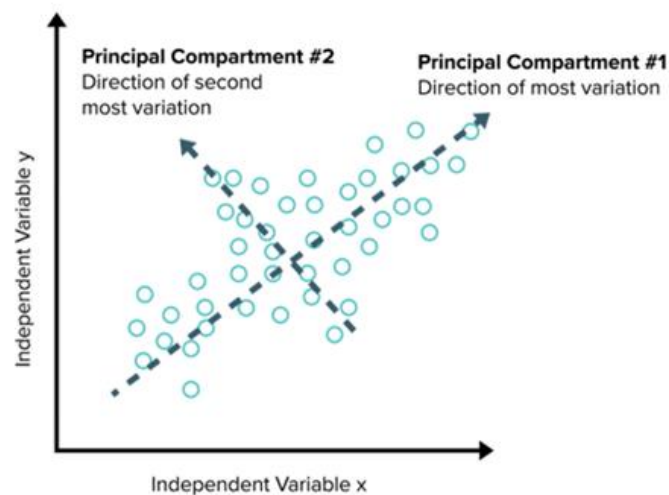
Logistic regression is a powerful fundamental machine learning method used for classification, but it is built on a regression framework.

Support vector machines (SVMs) are supervised learning models with associated learning algorithms that analyze data for classification and regression analysis.

Importing the Dataset and Exploratory Data Analysis (EDA)

The dataset is imported using the pandas library. The file is read using the .read_csv() by passing the name of the dataset as a string.

```
[ ]: import pandas as pd\nimport numpy as np\nimport matplotlib.pyplot as plt\nfrom sklearn.decomposition import PCA\nfrom sklearn.preprocessing import StandardScaler\nfrom sklearn.model_selection import train_test_split\nfrom sklearn.svm import SVC
```



Source: on, Originally published by Packt_Pub, "Principal Component Analysis-Unsupervised Learning Model," Hacker Noon, January 26, 2021.
<https://hackernoon.com/principal-component-analysis-unsupervised-learning-model-8f8c7683262>

Clustering

- **Unsupervised Training:** have only input vectors x , but no associated labels y
- **Goal:** identify "natural" groupings of the data
- **Inference:** for previous or new data point x decide cluster to which it belongs



Characteristics of clustering:

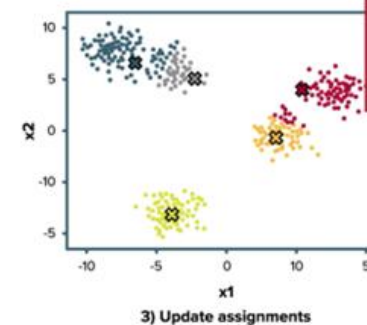
- Useful as part of a larger machine learning solution in combination with other machine learning methods
- Help determine representative labels for just few examples



Ex. Customer Segmentation

Clustering: k-means Algorithm

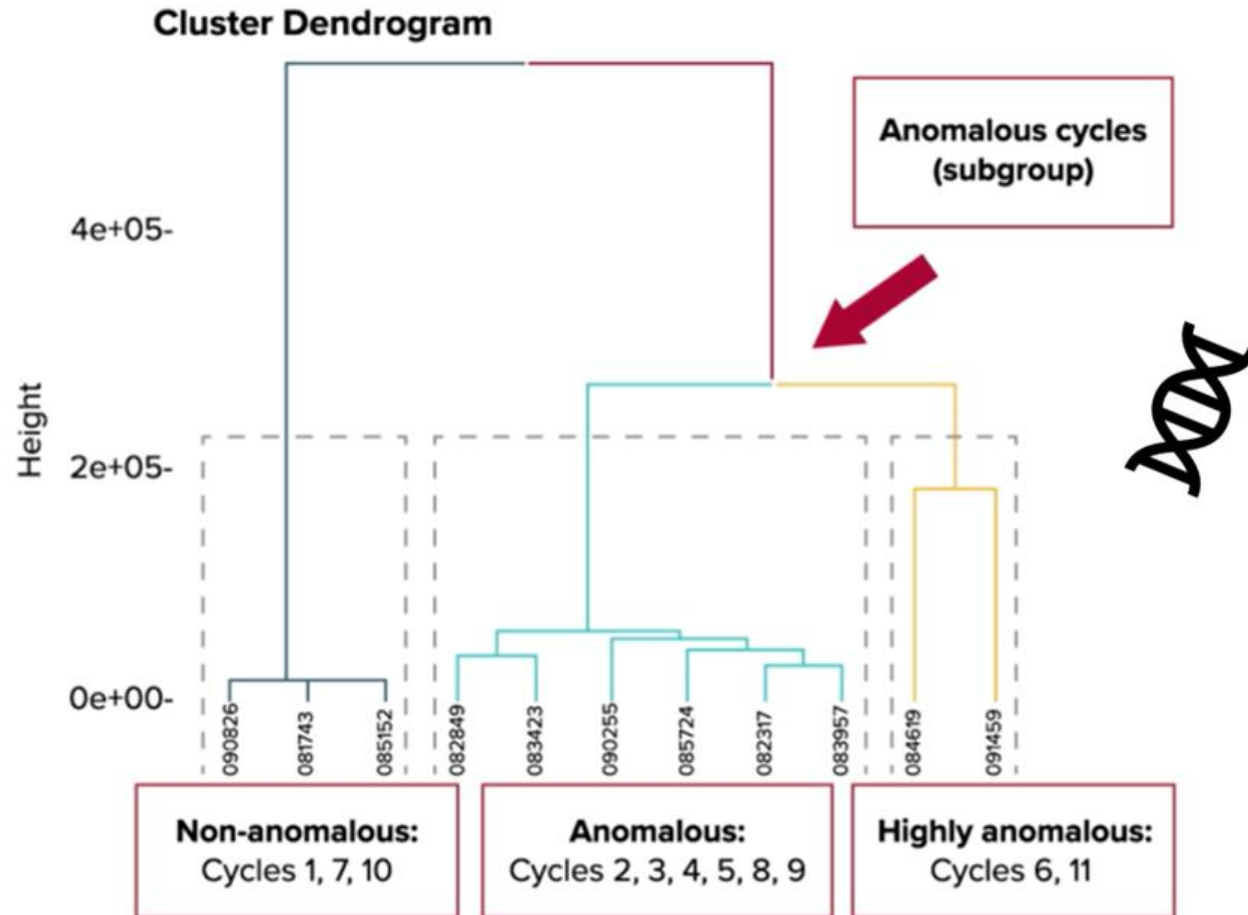
- Assume $m_1, \dots, m_k$ initial cluster means
- Use these means to assign each data point to the nearest cluster
- Adjust means m_i based on those memberships
- Iterate until some stopping criteria



Challenges in basic k -means:

- Assume k beforehand
- Additional approaches might be needed to sweep across different values of k

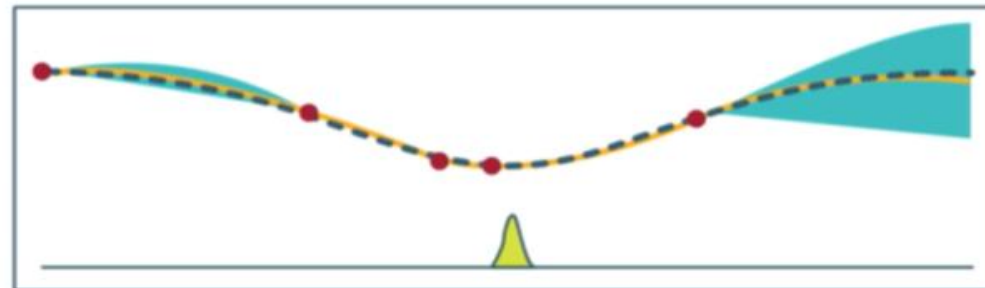
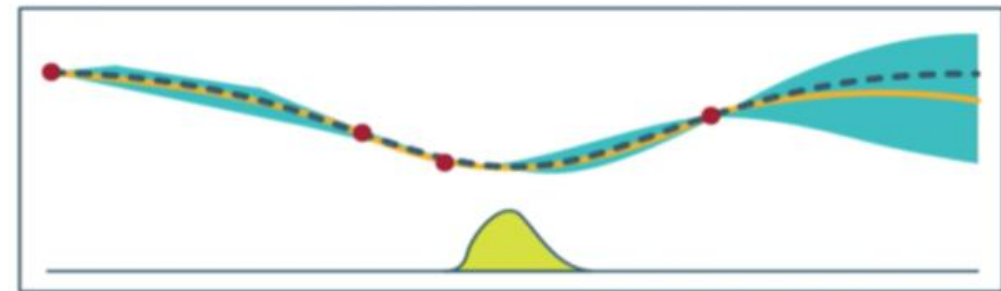
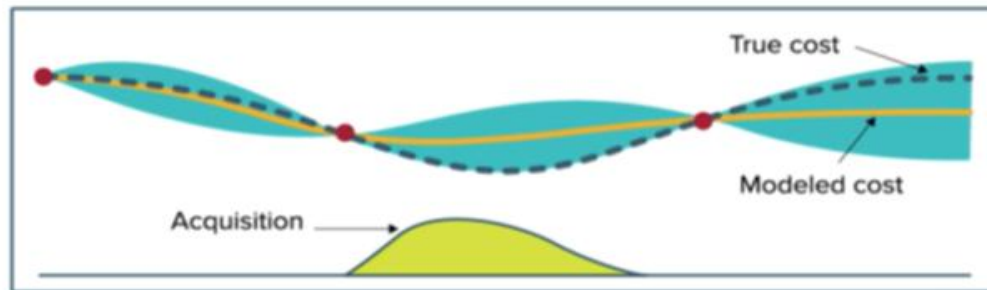
Hierarchical Clustering



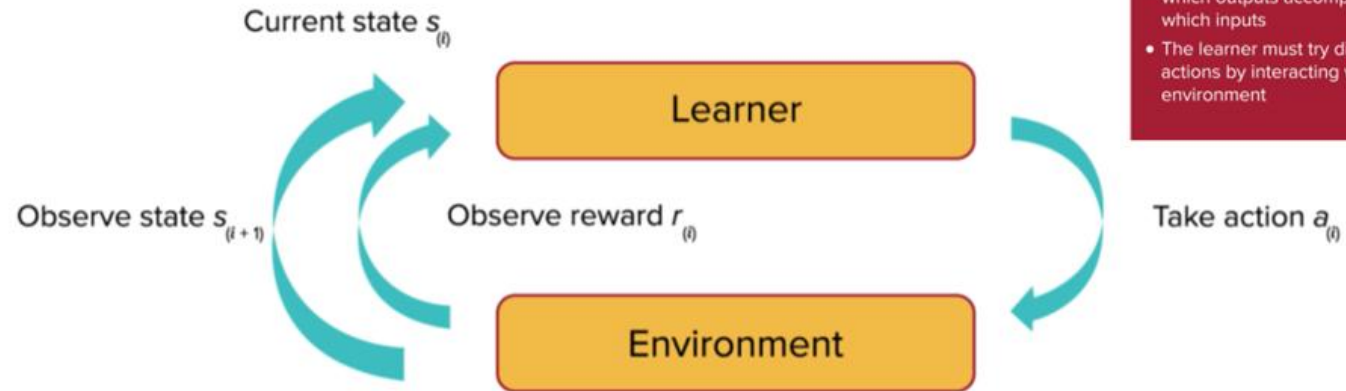
**Ex. Customer Service
Ticket Optimization**

MIT xPRO

Bayesian Optimization

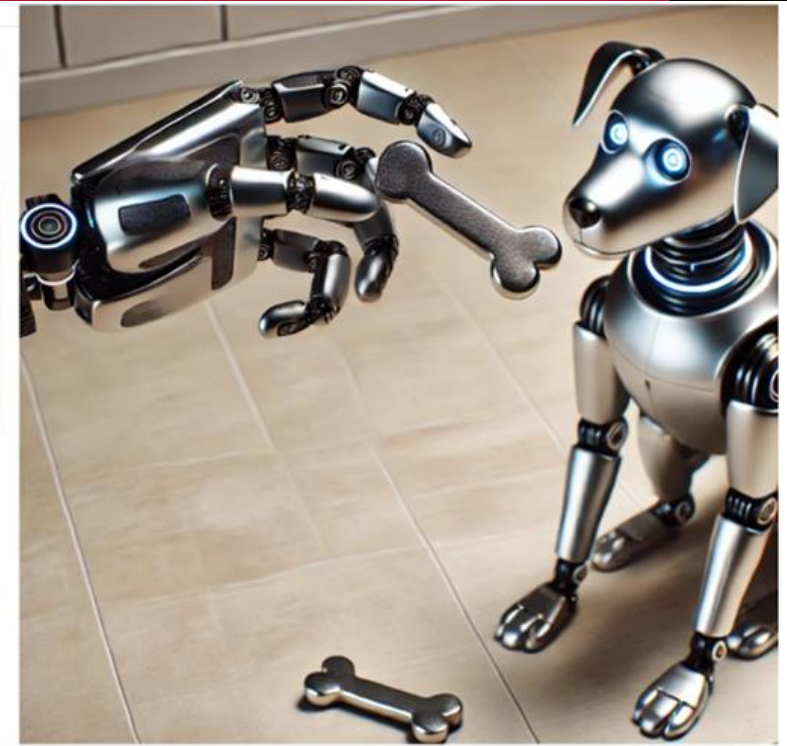


Reinforcement Learning



Reinforcement learning

- The learner is not told directly which outputs accompany which inputs
- The learner must try different actions by interacting with the environment



Ex. Training a Dog with Treats



Ex. Training a Robot to Navigate a Maze

The learner needs to:

- Explore this state-action space
- Build models of rewards
- Improve its knowledge and models

Classification vs. Regression Algorithms [Drag-and-Drop Activity 2.1]



Learning Outcome Addressed

1. Analyze the models, tools, and algorithms of machine learning.

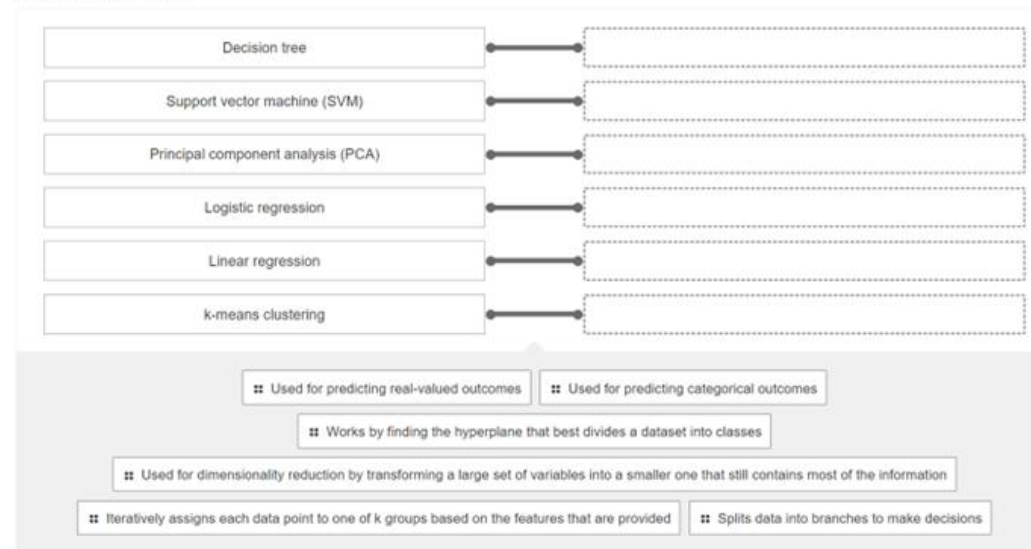
This is a self-study activity and does not count toward the completion of the program.

Task

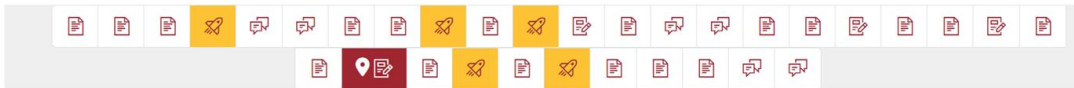
So far in this module, Professor Boning has covered classifiers and regression algorithms, each with its characteristics and purposes. In this activity, you'll differentiate between classification and a regression algorithm.

The average completion time for this activity is 10 minutes.

Drag the description from the left column to the correct ML algorithm in the right column. This activity will help you understand the differences between various algorithms. Below is the correct solution:



Assignment 2.4-Clustering



 Learning Outcome Addressed

4. Run machine learning algorithms in Python.

This is a self-study activity and does not count toward the completion of the program.

Task

You have learned about unsupervised machine learning modes. In this activity, you will practice running these algorithms in Python and analyzing the results.

Submission Instructions

The average completion time for this activity is 30 minutes.

- After opening the activity in EMOdE, complete and run the code following the instructions in the Jupyter Notebook Activity.
- Important: There is nothing to submit in EMOdE or Canvas for this practice assignment.
- Once you've tried the activity, you can view the solution in [Google Colab](#). Find a [guide to viewing your solution in Google Colab](#)  here.
 - Download the solution here: [Assignment 2.4-Clustering](#)

This tool needs to be loaded in a new browser window

Load Assignment 2.4-Clustering in a new window

Jupyter Assignment 2.4-Clustering Last Checkpoint: last month

File Edit View Run Kernel Nbgrader Settings Help

Python 3 (ipykernel) Validate

Assignment 2.4-Clustering

Activity overview

This activity is designed to consolidate the knowledge about k-means clustering and to define it in Python using the `sklearn` library.

A synthetic dataset from sklearn is used to visualize the clusters and choose an optimal number for the data they have.

This activity is designed to help learners apply the machine learning algorithms they have learned using packages in Python. Python concepts, instructions, and starter code are embedded within a Jupyter Notebook to guide them as they progress through the activity. The learner's should remember to run the code of each code cell prior to submitting the assignment. Upon completing the activity, the learners are encouraged to compare their work against the solution file to perform a self-assessment."

Basics of k-means

The k-means clustering method is an *unsupervised machine learning* technique used to identify clusters of data objects in a dataset. There are many different types of clustering methods, but k-means is one of the oldest and most approachable. These traits make implementing k-means clustering in `Python` reasonably straightforward, even for novice programmers and data scientists.

Clustering is a set of techniques used to partition data into groups, or clusters. Clusters are loosely defined as groups of data objects that are more similar to other objects in their cluster than they are to data objects in other clusters. In practice, clustering helps identify two qualities of data:

- **Meaningfulness** - find clusters to expand knowledge about the domain
- **Usefulness** - find clusters that can serve as an intermediate step in a data pipeline.

There are many applications of clustering, such as document clustering and social network analysis. Clustering applications are relevant in nearly every industry, making clustering a valuable skill for professionals working with data in any field.

Questions?

